
Entwicklung einer Android-App zur Akkuvorhersage mit TinyML

Portfolio-Dokumentation

Vertiefung I: Mobile und Ubiquitäre Anwendungen

Vorgelegt von: Keno Schürger

Matrikelnummer: 5023033

Studiengang: Wirtschaftsinformatik (BWI)

Dozent: Prof. Dr. Karsten Huffstadt

Semester: Sommersemester 2026



App direkt installieren
(Android, Sideload via Browser)

Hinweis zur App. Wer die App ohne Bauen-aus-dem-Quellcode direkt installieren möchte, kann den QR-Code scannen – das lädt die signierte Debug-APK (`HandyUsage-2026-05.apk`, ca. 22 MB) direkt aus dem GitHub-Repo aufs Handy. Beim ersten Start fragt Android nach der Erlaubnis zur Installation aus unbekannter Quelle. Nach dem Onboarding sammelt die App alle 30 Sekunden Sensor-Daten. Eine erste eigene Vorhersage erscheint nach 5 Minuten (zehn Messpunkte). Der Algorithmus-Tab funktioniert sofort und zeigt die Architektur des mitgelieferten Modells. Falls das Onboarding versehentlich weggeklickt wurde, lässt es sich jederzeit über das Zahnrad-Icon → „Onboarding noch einmal ansehen“ erneut aufrufen.

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	III
0 Ausgangssituation und Zielsetzung	1
0.1 Alleinstellungsmerkmal	1
0.2 Problemstellung	1
0.3 Konkrete Anwendungsszenarien	2
0.4 Mehrwert aus Nutzer- und Marktsicht	2
0.5 Zielsetzung	3
0.6 Rahmenbedingungen	4
0.7 Eckdaten des Projekts	4
1 Architekturmodell	5
1.1 Was passiert im Inneren	5
1.2 Gesamtarchitektur im Überblick	5
1.3 Android-App im Detail	8
1.4 Die App in Aktion – Bedienungsanleitung mit Screenshots	9
1.5 Python-Pipeline im Detail	18
2 Datenmodell	19
2.1 CSV-Schema	19
2.2 Warum genau diese zehn Features?	19
2.3 Speicherung und Export	20
2.4 Datenschutz	20
3 Programmierung	23
3.1 Datensammlung im Service	23
3.2 Auslesen der 10 Features	24
3.3 On-Device-Inferenz mit dem TinyML-Modell	25
3.4 Robustheit – die App muss überleben	26
3.5 Schema-Drift im Logger	27
3.6 Python-Pipeline – Beispiel Segment-Erkennung	28
3.7 Trainings-Verlauf	28
3.8 Code-Umfang und Eigenleistung	29
3.9 Reproduzierbarkeit der Pipeline	30
4 Abschließender Erfahrungsbericht	31
4.1 Wöchentliche Einträge	31

4.2	Schlüssel-Entscheidungen, Fehler und Trade-offs	33
4.3	KI-Nutzung	34
Literatur		37

Abbildungsverzeichnis

1	Vereinfachter Datenfluss zwischen den Systemkomponenten	6
2	Detaillierte Gesamtarchitektur des Battery-Predictor-Systems	7
3	Onboarding-Slides 1 und 4. Slides 2 und 3 dazwischen erklären Sensoren und PC-Training.	10
4	Live-Tab im Zustand „Lädt“: das Label oben („STATUS“) wechselt dynamisch, damit nicht „AKKU NOCH Lädt“ entsteht. Confidence-Pille ist in diesem Zustand bewusst ausgeblendet (Vorhersage pausiert).	11
5	Algorithmus-Tab, obere zwei Visualisierungen.	12
6	Algorithmus-Tab, untere Sektionen: Live-Sparkline und USP-Pillen.	13
7	Genauigkeits-Tab: für den letzten beobachteten Discharge-Lauf wird der MAE der drei Methoden (eigenes TinyML, Google API, Linear-Baseline) verglichen. Hier auf dem Xiaomi: alle drei „>10 h daneben“ – bewusst ehrliche Anzeige eines Falls, in dem das Modell nicht überzeugt.	14
8	Settings-Activity. Hinweis: der Button „Onboarding noch einmal ansehen“ ist exakt für den Fall gedacht, dass der Prüfer das Onboarding versehentlich weggeklickt hat.	15
9	Train-Own-Model-Activity, aufgerufen über den Settings-Button „Modell selbst trainieren“.	16
10	Trainings- und Validation-MAE (durchschnittliche Abweichung in Stunden) des TinyML-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5 h. Da EarlyStopping mit <code>restore_best_weights=True</code> konfiguriert ist, wird das beste Modell zurückgegeben.	29

Tabellenverzeichnis

1	Projekt auf einen Blick	4
2	Module der Android-App	8
3	Felder eines CSV-Eintrags	19
4	Im Manifest deklarierte Permissions	21
5	Berechtigungen mit Nutzer-Aktion	21
6	Code-Umfang nach Bereich	29
7	Schlüssel-Entscheidungen und verworfene Alternativen	34

Quellcode-Verzeichnis

1	Mess-Schleife im DataCollectorService	23
---	---	----

2	CPU-Last als Frequenz-Proxy	24
3	App-Kategorisierung (Auszug)	24
4	Hotspot-Status via Reflection	25
5	Inferenz mit TFLite	25
6	Auto-Restart bei onTaskRemoved	27
7	Schema-Drift im Logger abfangen	27
8	Discharge-Segmente finden	28
9	Pipeline reproduzierbar starten	30

0 Ausgangssituation und Zielsetzung

0.1 Alleinstellungsmerkmal

Eine eigene Akku-Vorhersage durch ein 14,4 KB großes TinyML-Modell, das vollständig offline auf jedem Android-Gerät läuft.

Das ist das Hauptfeature dieser Arbeit. Alle weiteren Entscheidungen – von der Sensorauswahl über die Modell-Architektur bis zur Bedienoberfläche – ordnen sich ihm unter.

Das mitgelieferte TinyML-Modell ist mit 14,4 KB kleiner als eine WhatsApp-Sprachnachricht. Es läuft INT8-quantisiert auf jeder Android-Version ohne spürbaren Energie- oder Speicher-Overhead, und alle Sensordaten bleiben dabei lokal auf dem Gerät. Es gibt keinen Server-Roundtrip.

Jeder Nutzer kann das Modell mit eigenen Daten nachtrainieren. Dafür sammelt er die Daten in der App, trainiert am PC ein neues Modell und deployt die fertige TFLite-Datei zurück in die App. So lässt sich das Modell auf sein konkretes Gerät und sein Nutzungsverhalten zuschneiden. Die App ist damit nicht nur Konsument eines vortrainierten Modells, sondern Teil einer schließbaren Trainings-Schleife.

Auf älteren Geräten ist die App *überhaupt* die einzige verfügbare ML-Vorhersage: erst mit Android 12 (Oktober 2021) hat Google eine ML-basierte Akku-Schätzung ins System eingebaut – alle Android-Versionen davor (also über zehn Jahre Android-Geräte) bieten nur die einfache lineare Stundenanzeige. Auf einem Smartphone, das nie ein Android 12-Update bekommen hat, ist diese App damit die einzige Möglichkeit, eine neuronale Restlaufzeit-Vorhersage zu bekommen.

0.2 Problemstellung

Smartphone-Akkuanzeigen funktionieren auf den meisten Geräten nach einem einfachen Prinzip: man rechnet den durchschnittlichen Verbrauch der letzten Stunden linear in die Zukunft. Wer schon einmal mit 30 % Akku „noch 4 Stunden“ im Display hatte und dann nach einem YouTube-Video plötzlich bei 15 % mit „noch 1 Stunde“ stand, kennt das Problem.

Akku-Verbrauch ist in Wahrheit nichtlinear und vom Kontext getrieben: Streaming verbraucht ein Vielfaches von Lesen, Navigation in der Kälte mehr als bei Wärme, eine schwache Mobilfunk-Verbindung mehr als WLAN, ein heißes Spiel mehr als der Sperrbildschirm. Ein linearer Durchschnitt der letzten Minuten kann diese Dynamik prinzipbedingt nicht abbilden – dafür braucht es ein Modell, das die Beziehung zwischen Sensoren und Verbrauch tatsächlich *gelernt* hat.

Genau das ist auf den meisten Smartphones bis heute nicht vorhanden: erst mit Android 12 (Oktober 2021) hat Google überhaupt ein eigenes ML-Modell ins System eingebaut (`PowerManager.getBatteryDischargePrediction()`) [2]. Für alle Android-Versionen davor – und damit für einen großen Teil der weltweit aktiven Geräte – gibt es weiterhin nur die einfache lineare Schätzung. Eine echte, an das eigene Nutzungsverhalten angepasste KI-Vorhersage existiert dort schlicht nicht.

0.3 Konkrete Anwendungsszenarien

Damit klar wird, für wen das überhaupt relevant ist – vier Alltagssituationen, in denen eine verlässliche Restlaufzeit messbar hilft. Allen gemeinsam: die nächste Steckdose ist eben nicht in fünf Minuten erreichbar.

- **Beim Wandern oder auf Mehrtagestouren.** Eine verlässliche Restlaufzeit entscheidet, ob man unterwegs noch die Karte öffnen kann oder lieber den Flugmodus aktiviert. Genau das, was ein gelerntes Modell besser hinbekommt als ein linearer Durchschnitt.
- **Auf Reisen ohne sichere Ladequelle.** Im Zug, im Flugzeug, im Hotelzimmer ohne freie Steckdose. Statt nervös alle zehn Minuten zu prüfen, gibt eine verlässliche Schätzung Planungssicherheit (z. B. ob man Filme bis zur Ankunft schauen kann).
- **Im Pendel-Alltag.** „Reicht der Akku noch für den Heimweg, oder muss ich im Büro noch laden?“ Eine Restlaufzeit am Vormittag erlaubt die richtige Entscheidung am Mittag.
- **Beim aktiven Tagesplanen.** Wer das Handy für Navigation, Musik und Foto braucht (Konzert, Festival, Stadtführung), nutzt das Gerät wesentlich aggressiver als an einem normalen Büroarbeitstag. Ein Modell, das die zehn Sensor-Features in genau diesem Verhaltensmuster gelernt hat, ist hier deutlich nützlicher als ein generischer Durchschnitt.

Allen Szenarien gemeinsam: seltener auf die Akku-Anzeige schauen müssen, weniger ungeplante Ladestopps, mehr Planungssicherheit. Genau diese vier Situationen sind als Slide 1 im Onboarding der App hinterlegt – der Nutzer sieht sie beim ersten Öffnen sofort.

0.4 Mehrwert aus Nutzer- und Marktsicht

Aus Nutzer- und Marktsicht steht die App auf fünf Punkten:

- **Eigenes Modell, kein API-Wrapper.** Kern ist ein selbst trainiertes TinyML-Modell, das auf dem Gerät läuft – nicht eine reine UI-Schicht über einer vorhandenen

API. Das unterscheidet die App von typischen Akku-Optimierer-Apps aus dem Play Store.

- **Auf das eigene Gerät zuschneidbar.** Das mitgelieferte Modell läuft auf jedem Android-Gerät als Basis; jeder Nutzer kann es darüber hinaus auf den eigenen gesammelten Daten nachtrainieren und so auf die Eigenheiten seines Geräts und seines Nutzungsverhaltens zuschneiden. Eine generische System-Schätzung oder Cloud-API hat diese Möglichkeit nicht – dort bekommt jedes Gerät dieselbe Vorhersage-Logik.
- **Direkter Alltagsnutzen.** Die vier Szenarien aus Abschnitt 0.3 (Wandern, Reisen, Pendeln, Tagesplanen) zeigen konkret, wo eine verlässliche Vorhersage hilft.
- **Keine laufenden Kosten.** Inferenz läuft komplett auf dem Endgerät. Eine Verteilung an 1.000 oder 10.000 Nutzer kostet dasselbe wie an einen einzigen: nichts. Kein Cloud-Compute, kein API-Call pro Vorhersage, keine variable Infrastruktur in der Kostenrechnung.
- **Niedrige Einstiegshürde.** Kein Account, kein Abo, keine Cloud-Anmeldung. Onboarding in unter 90 Sekunden, direkt danach die erste Vorhersage.

0.5 Zielsetzung

Ziel der Arbeit ist die Umsetzung des in Abschnitt 0.1 beschriebenen Hauptfeatures innerhalb einer Android-App: eine eigene Restlaufzeit-Vorhersage durch ein TinyML-Modell, das vollständig auf dem Gerät läuft und vom Nutzer auf den eigenen gesammelten Daten nachtrainiert werden kann.

Damit dieses Hauptfeature belastbar wird, müssen vier Bausteine zusammenspielen:

1. Eine Hintergrund-Datensammlung, die alle 30 Sekunden zehn Sensor-Features aufzeichnet (Akku, Helligkeit, App-Kategorie, CPU, Temperatur, Netzwerk-Zustand, ...) und über einen Foreground-Service auch Geräte-Neustarts und das Wegwischen aus der Recent-Apps-Übersicht übersteht. Ohne saubere Datensammlung kein verlässliches Modell.
2. Ein quantisiertes TFLite-Modell (< 15 KB), das in der App on-device läuft – klein genug, um auch auf älteren Smartphones flüssig zu inferieren, und klein genug, um es mit jeder neuen Trainings-Runde durch eine neue Version ersetzen zu können. Vorhersage erscheint in Notification und in drei spezialisierten Tabs (Live, Algorithmus, Genauigkeit).
3. Eine Python-Trainings-Pipeline am PC, die die aus der App exportierte CSV ein-

liest, daraus ein nutzerspezifisches Modell trainiert und es als neue `.tflite`-Datei zurück in die App deployt. Das mitgelieferte Modell läuft schon direkt nach Installation. Die Pipeline ist ein optionaler Extra-Schritt für Nutzer, die das Modell auf ihre eigenen Daten zuschneiden möchten.

4. Eine eingebaute Auswertung, die im Genauigkeits-Tab live zeigt, wie das eigene Modell gegen die Google-API und eine einfache lineare Baseline abschneidet.

0.6 Rahmenbedingungen

- **Eigenes Gerät:** Xiaomi 2107113SG (Redmi Note 10 Pro), Android 12, 8 GB RAM.
- **Zusätzliche Test-Geräte** (Familie): Google Pixel 7 Pro, Pixel 8 Pro, Pixel 9 Pro XL.
- **Entwicklungsumgebung:** Android Studio (Kotlin), Python 3.11 mit TensorFlow 2.14 am Windows-PC.

0.7 Eckdaten des Projekts

Das Projekt umfasst am Ende drei Schichten: die Android-App, eine Python-Pipeline für Training und Auswertung sowie das zurück-deployte TinyML-Modell.

Tabelle 1: Projekt auf einen Blick

Kennzahl	Wert
Code-Umfang gesamt	≈ 5.600 Zeilen (Kotlin + Python + XML)
Android-App	20 Kotlin-Module (Service, Inferenz, UI, Onboarding, Settings)
Python-Pipeline	25 Module in zwei Schichten (<code>methods/</code> , <code>evaluation/</code>)
Test-Geräte	4 (Xiaomi 2107113SG + Pixel 7 / 8 / 9 Pro, davon 3 bei Familienmitgliedern)
Feld-Sammlung	48 Tage Dauerbetrieb, 180 Service-Sessions
Gesammelte Messpunkte	66.001×10 Features
TinyML-Modell	14,4 KB INT8-quantisiert, $\sim 5 \mu\text{s}$ pro Inferenz

1 Architekturmodell

1.1 Was passiert im Inneren

Die App arbeitet in drei aufeinander aufbauenden Schritten:

1. **Daten sammeln.** Alle 30 Sekunden schreibt die App zehn Werte vom Handy in eine CSV-Datei – aktueller Akkustand, Bildschirm-Helligkeit, aktive App-Kategorie (z. B. Video, Browser, Spiel), Geräte-Temperatur, CPU-Last, Netzwerk-Zustand und so weiter.
2. **Modell trainieren am PC.** Die Tabelle wird auf den PC übertragen und dort von einem neuronalen Netz (Conv1D + Dense) durchgearbeitet. In jeder Trainings-Iteration vergleicht das Modell seine vorhergesagte Restlaufzeit mit dem tatsächlichen Akku-Verlauf in der Tabelle und justiert seine internen Gewichte so, dass die Differenz schrumpft. Nach wenigen Minuten hat es Zusammenhänge gelernt wie: *Helligkeit hoch, Video-App aktiv, Temperatur 38°C → Restlaufzeit etwa 2 Stunden.*
3. **Vorhersagen im Alltag.** Das fertig trainierte Modell ist nur 14,4KB groß – kleiner als ein WhatsApp-Sprachmemo. Es wird als `.tflite`-Datei zurück in die App deployt, läuft dort offline und liefert alle 30 Sekunden eine neue Restlaufzeit.

Warum genau diese Aufteilung? Die drei Schritte sind absichtlich auf zwei Geräte aufgeteilt (Handy für Schritte 1 und 3, PC für Schritt 2). Das hat zwei Gründe:

- Auf dem Handy ist das Trainieren des Modells zu rechenintensiv – es würde den Akku stark belasten und Stunden dauern. Auf dem PC ist es in wenigen Minuten erledigt.
- Das fertige Modell ist dafür so klein, dass es im Handy problemlos im Hintergrund läuft, ohne dass der Nutzer etwas davon merkt. *Trainieren ist teuer, vorhersagen ist billig* – diese Asymmetrie macht den On-Device-Einsatz überhaupt erst möglich.

1.2 Gesamtarchitektur im Überblick

Die drei Phasen aus Abschnitt 1.1 verteilen sich auf zwei Geräte-Welten (Android-App und PC-Pipeline), die über CSV-Dateien und das TinyML-Modell als Schnittstellen miteinander kommunizieren. Abbildung 1 zeigt den Datenfluss vereinfacht, Abbildung 2 auf der folgenden Seite die vollständige Detail-Architektur mit allen Unterkomponenten und internen Verbindungen.

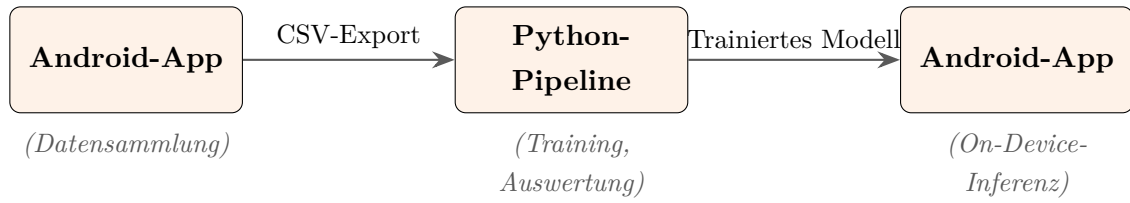


Abbildung 1: Vereinfachter Datenfluss zwischen den Systemkomponenten

Gesamtarchitektur: Akkuvorhersage mit TinyML

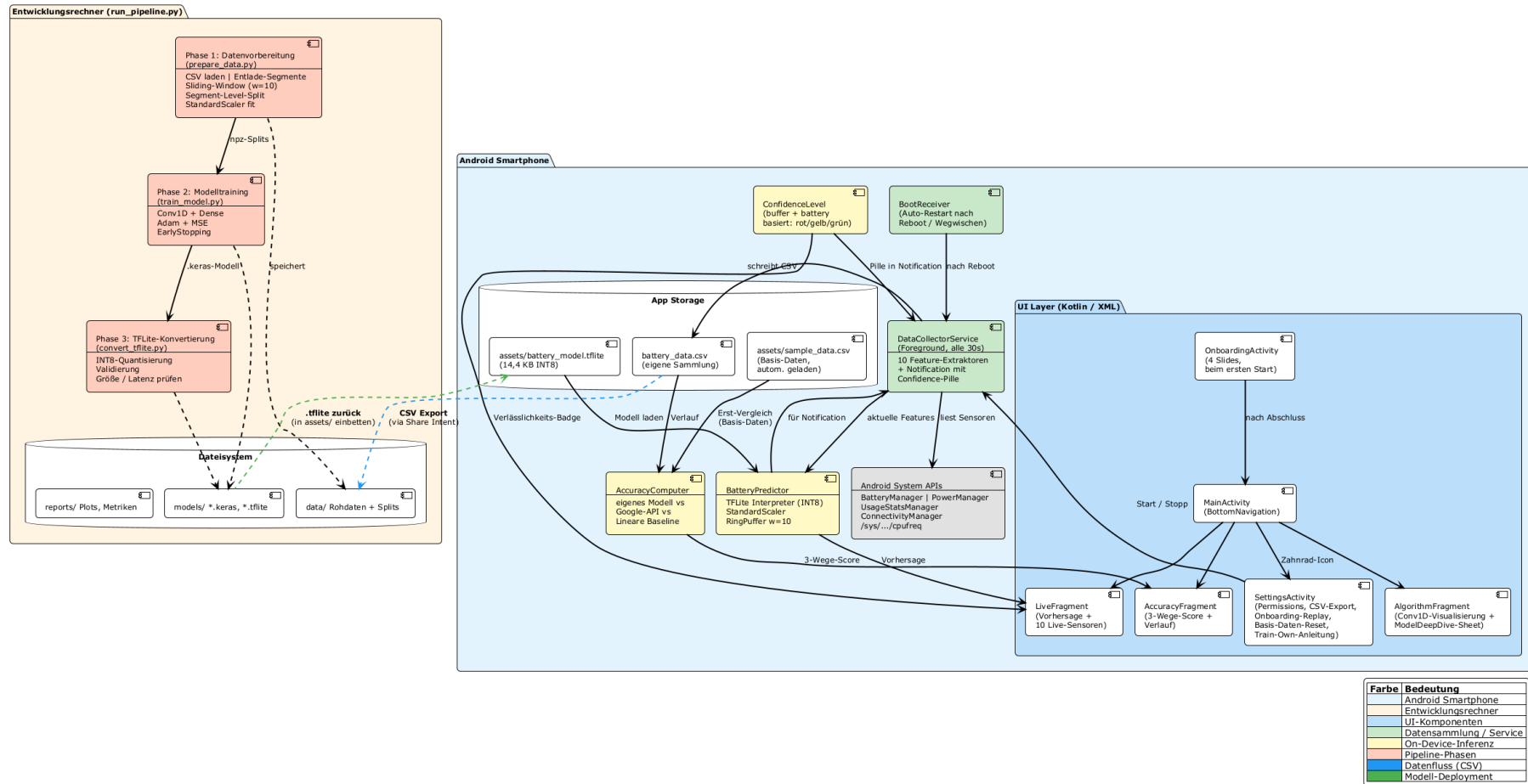


Abbildung 2: Detaillierte Gesamtarchitektur des Battery-Predictor-Systems

1.3 Android-App im Detail

Die App ist in zwanzig Kotlin-Dateien gegliedert, sauber getrennt in App-Lifecycle, Datenerhebungs-Service, Modell-Inferenz und UI-Bausteine:

Tabelle 2: Module der Android-App

Datei	Aufgabe
<i>Datenerhebung und Modell</i>	
DataCollectorService.kt	Foreground-Service, sammelt alle 30 Sekunden Daten und sendet Broadcasts an die UI
BatteryDataPoint.kt	Datenklasse für einen Messpunkt mit CSV-Serialisierung
BatteryDataLogger.kt	Schreibt CSV in filesDir, behandelt Schema-Drift und Legacy-Migration
BatteryPredictor.kt	TFLite-Interpreter mit StandardScaler-Normalisierung
BootReceiver.kt	Startet den Service nach Geräte-Neustart
<i>App-Lifecycle und Onboarding</i>	
MainActivity.kt	Tab-Container mit Bottom-Navigation, Settings-Icon und Onboarding-Routing beim Erststart
OnboardingActivity.kt	Vier-Slide-Erstkontakt: Anwendungsszenarien, Sammeln-Lernen-Vorhersagen-Prinzip und Modell-Größen-Vergleich (14,4 KB)
OnboardingPagerAdapter.kt	ViewPager2-Adapter für die Onboarding-Slides
<i>UI-Tabs</i>	
LiveFragment.kt	Prominente Vorhersage-Anzeige, Confidence-Pille und Live-Anzeige der 10 Sensor-Werte als animierte Karten
AlgorithmFragment.kt	Erklärt in vier Schritten, wie aus Sensoren die Vorhersage wird, und bindet die drei Custom Canvas-Views ein
DataflowView.kt, BatteryHistoryChart.kt, ConvLayerView.kt	Drei Custom Canvas-Views: animierter Sensor-zu-Vorhersage-Datenfluss, Sparkline-Akkuverlauf mit Vorhersage-Extension und Conv1D-Modell-Architektur (Details siehe Abschnitt 1.4)
ModelDeepDiveSheet.kt	Bottom-Sheet im Algorithmus-Tab: zeigt bei Tap auf die Modell-Architektur die ausführliche Erklärung
AccuracyFragment.kt	Genauigkeits-Tab mit 3-Wege-Score (TinyML / Google / Linear)

Fortsetzung auf nächster Seite

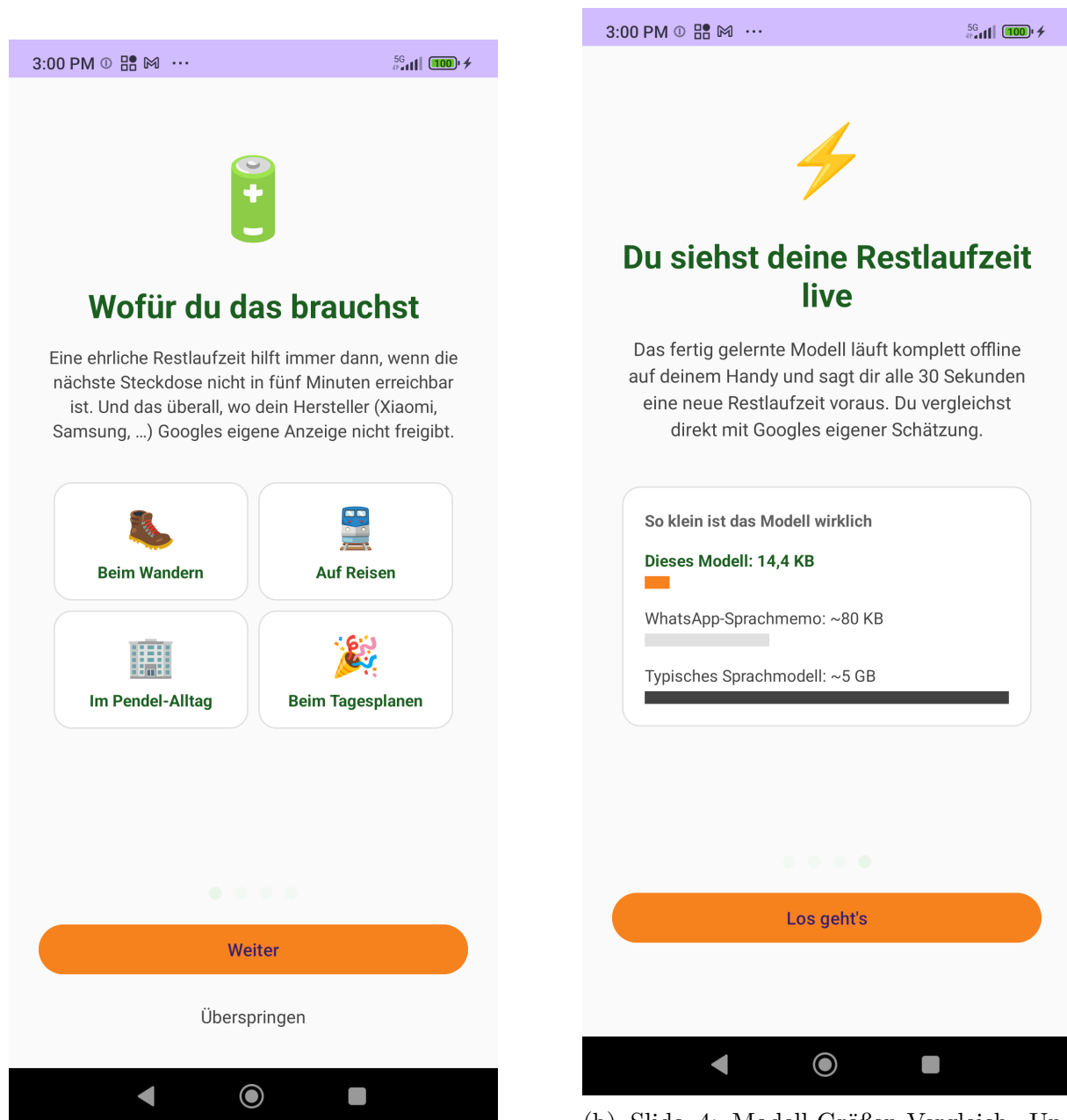
Fortsetzung Tabelle 2

Datei	Aufgabe
<code>AccuracyComputer.kt</code>	Liest die CSV-Daten und berechnet den MAE der drei Methoden im letzten Discharge-Lauf
<i>Infrastruktur-UI</i>	
<code>SettingsActivity.kt</code>	Hinter dem Zahnrad-Icon: Service-Steuerung, Permissions, CSV-Export, Modell-Info und mehrere Sub-Aktionen (Onboarding-Replay, Basis-Daten-Reset, Train-Own-Anleitung)
<code>TrainOwnModelActivity.kt</code>	Schritt-für-Schritt-Anleitung, wie der Nutzer ein eigenes Modell auf seinen Daten trainieren und in die App deployen kann
<code>ConfidenceBadge.kt</code>	Wiederverwendbarer Confidence-Chip mit Farbkodierung (grün / gelb / rot)
<code>ConfidenceLevel.kt</code>	Enum + Heuristik (Buffer-Größe + Akkustand)

1.4 Die App in Aktion – Bedienungsanleitung mit Screenshots

Diese Sektion gibt eine vollständige Bedienungsanleitung der App und ergänzt die Modul-Beschreibung in Tabelle 2 um Screenshots aller relevanten Bildschirme.

Welcome-Bildschirm: Onboarding (vier Slides). Beim allerersten Öffnen wird automatisch das vierteilige Onboarding gezeigt. Slide 1 verortet den Nutzen anhand vier Alltagssituationen, Slide 2–3 erklären das „Sammeln → Lernen → Vorhersagen“-Prinzip in einfachen Worten, Slide 4 zeigt den Modell-Größen-Vergleich. Das Onboarding ist später jederzeit über das Zahnrad-Icon → „Onboarding noch einmal ansehen“ wieder aufrufbar.



(a) Slide 1: Vier Anwendungsszenarien als 2×2-Grid (Wandern, Reisen, Pendel-Alltag, Tagesplanen).

(b) Slide 4: Modell-Größen-Vergleich. Unser Modell (14,4 KB) in Orange, daneben WhatsApp-Sprachmemo und ein typisches Sprachmodell.

Abbildung 3: Onboarding-Slides 1 und 4. Slides 2 und 3 dazwischen erklären Sensoren und PC-Training.

Tab „Live“: das Hauptfeature. Direkt nach dem Onboarding (oder bei jedem späteren Öffnen) landet der Nutzer im Live-Tab. Oben groß die Restlaufzeit (oder ein expliziter Status „Lädt“ / „Voll“ mit passender Sub-Erklärung), darunter alle zehn Live-Sensoren als animierte Karten – die Hauptbühne der App.

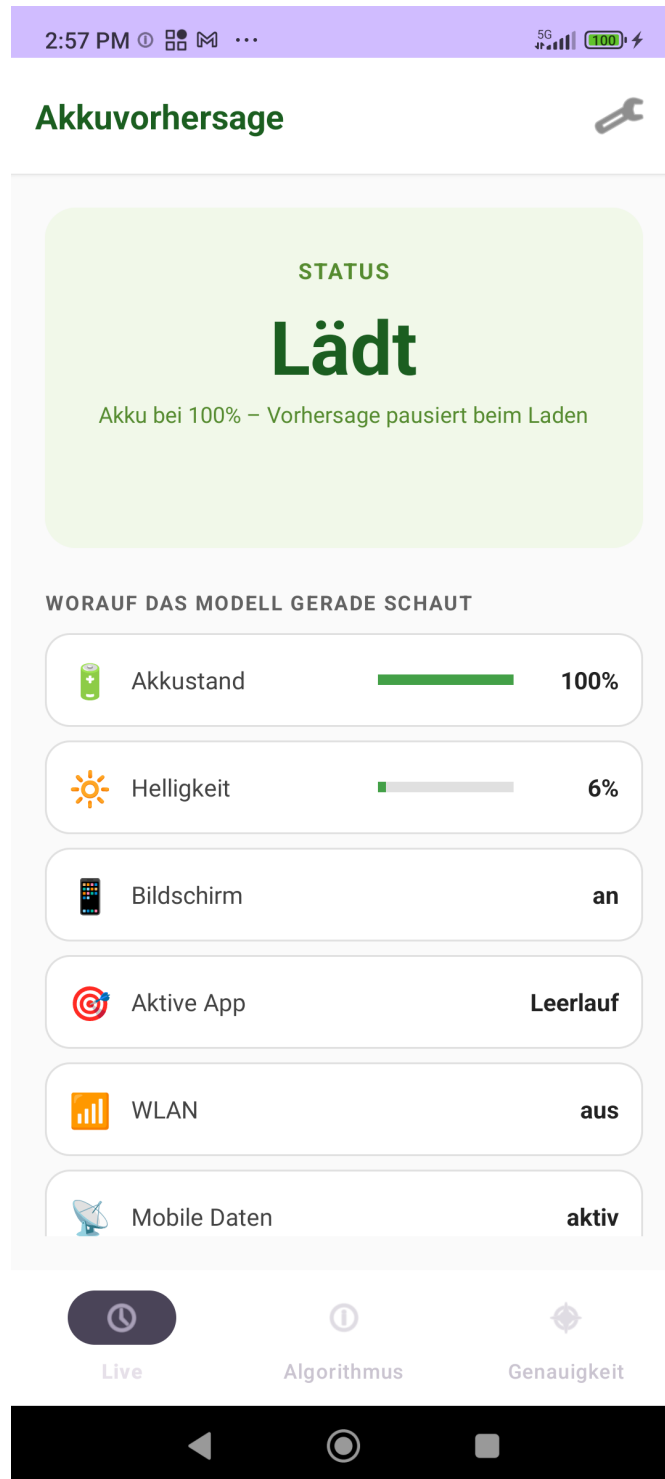
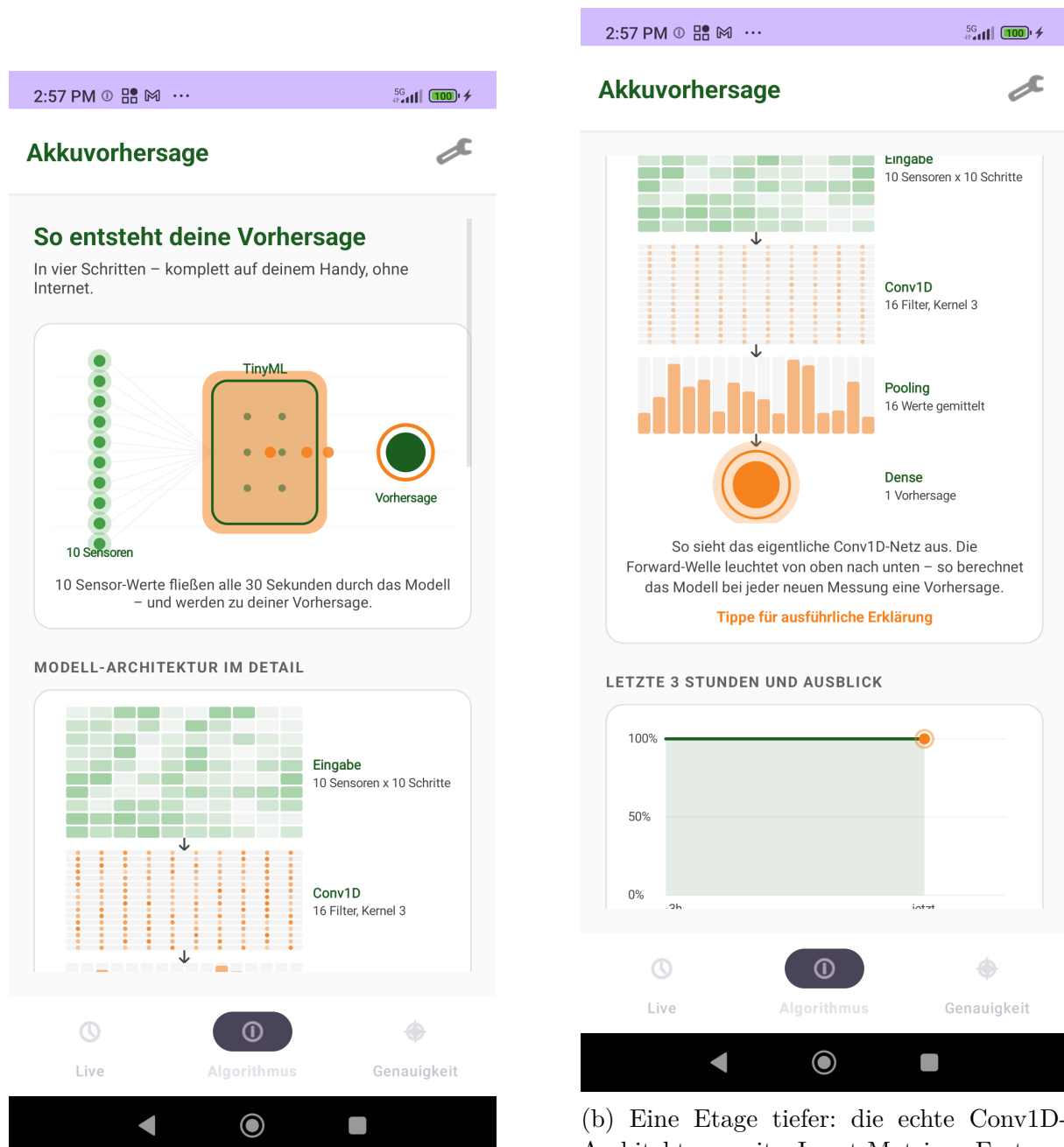


Abbildung 4: Live-Tab im Zustand „Lädt“: das Label oben („STATUS“) wechselt dynamisch, damit nicht „AKKU NOCH Lädt“ entsteht. Confidence-Pille ist in diesem Zustand bewusst ausgeblendet (Vorhersage pausiert).

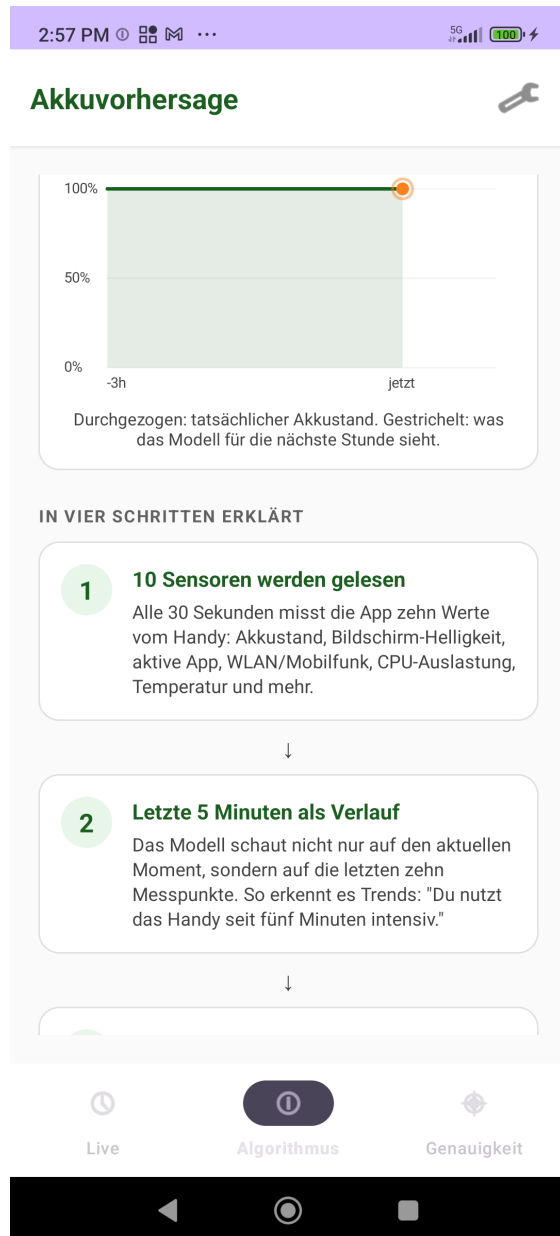
Tab „Algorithmus“: die drei Visualisierungen. Der mittlere Tab erklärt, wie der Algorithmus tatsächlich funktioniert. Drei eigene Custom-Canvas-Views, die alle live laufen.



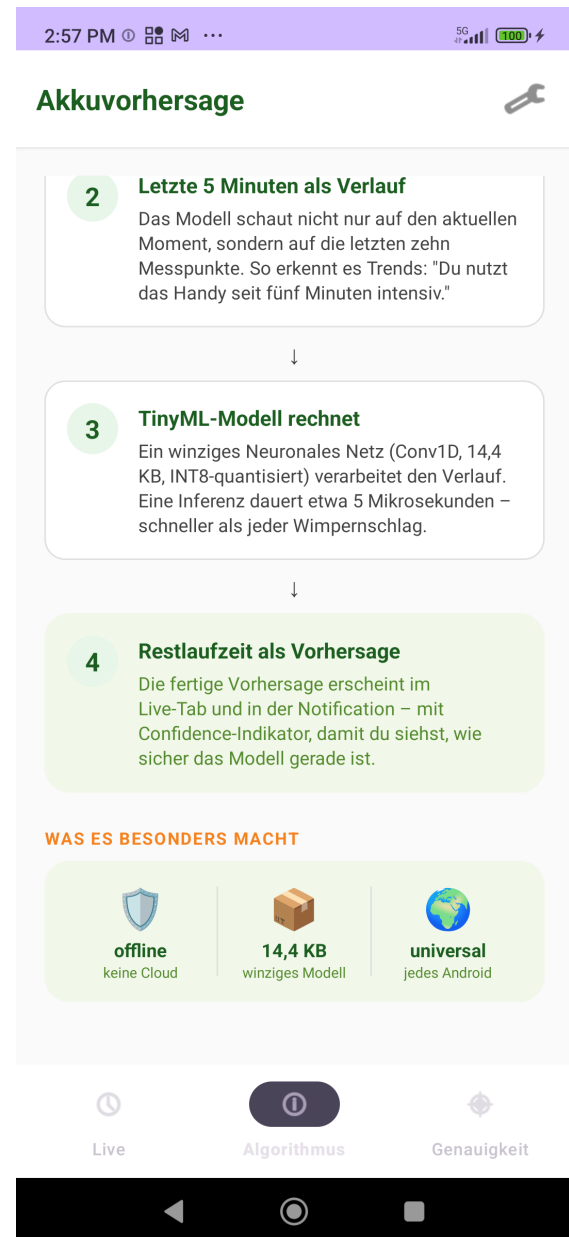
(a) Oben: animierter Datenfluss von zehn pulsierenden Sensoren durch die TinyML-Box zur Vorhersage rechts.

(b) Eine Etage tiefer: die echte Conv1D-Architektur mit Input-Matrix, Feature-Maps, Pooling und Dense-Output. Eine Forward-Pass-Welle leuchtet von oben nach unten.

Abbildung 5: Algorithmus-Tab, obere zwei Visualisierungen.



(a) Live-Sparkline der letzten drei Stunden Akkuverlauf plus gestrichelte Vorhersage-Extension. Hier (Akku zu 100%) nur als Linie sichtbar, weil keine Discharge-Dynamik vorhanden.



(b) Ganz unten der USP-Block „Was es besonders macht“: offline, 14,4 KB, universal.

Abbildung 6: Algorithmus-Tab, untere Sektionen: Live-Sparkline und USP-Pillen.

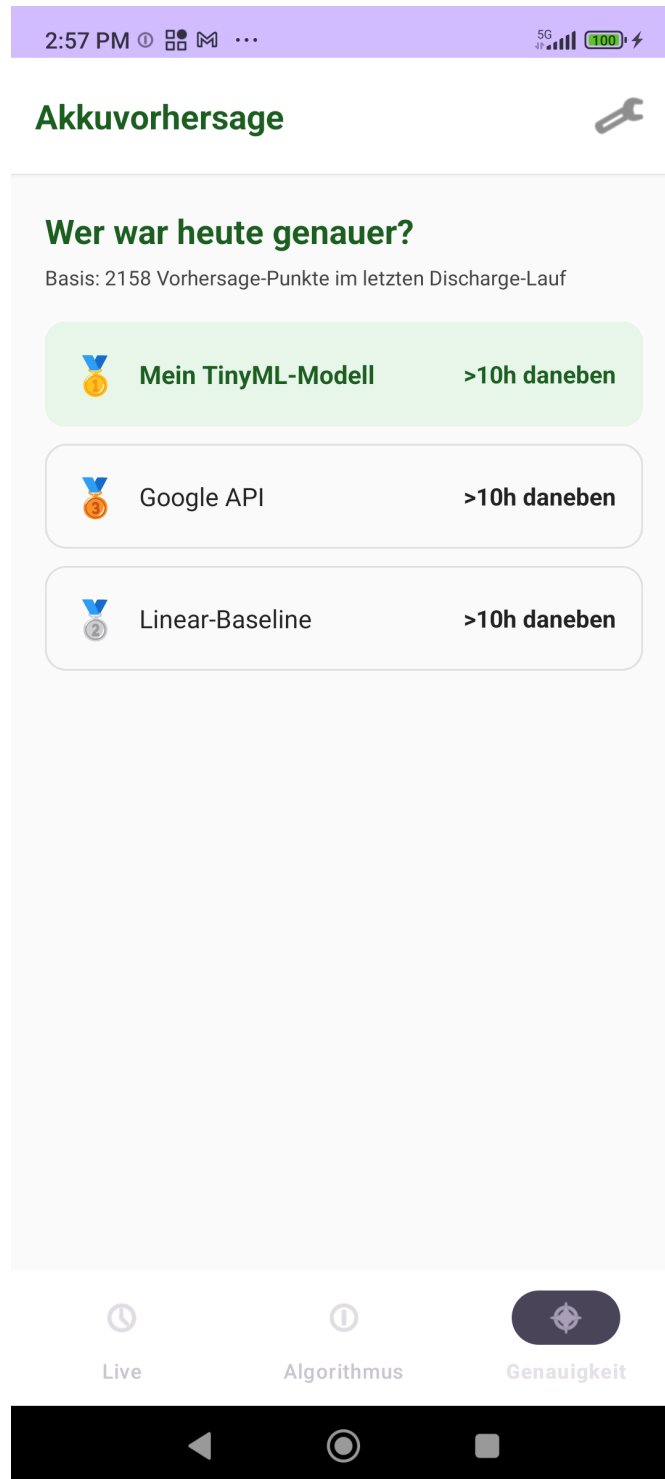
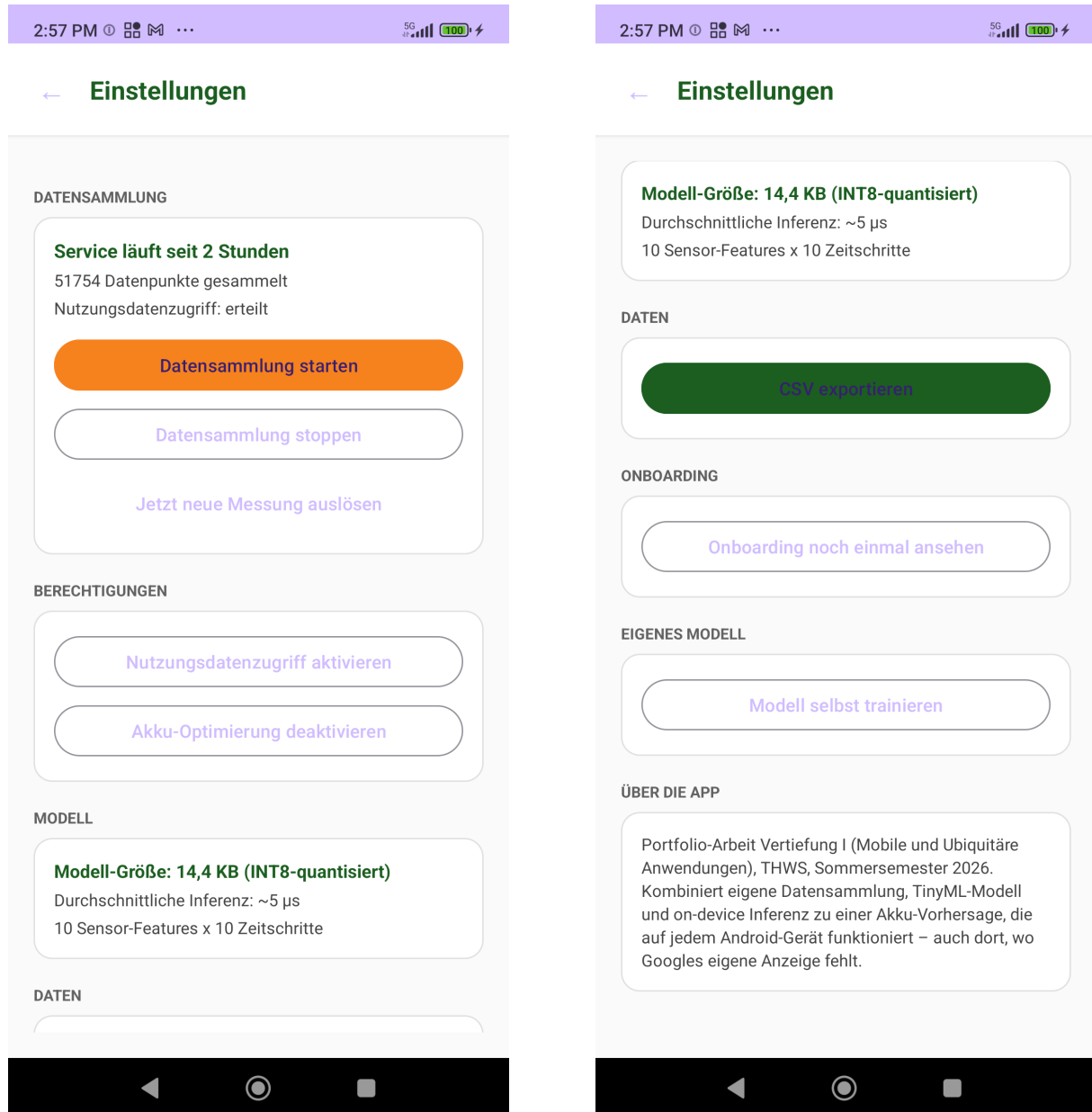


Abbildung 7: Genauigkeits-Tab: für den letzten beobachteten Discharge-Lauf wird der MAE der drei Methoden (eigenes TinyML, Google API, Linear-Baseline) verglichen. Hier auf dem Xiaomi: alle drei „>10 h daneben“ – bewusst ehrliche Anzeige eines Falls, in dem das Modell nicht überzeugt.

Tab „Genauigkeit“: der ehrliche Vergleich.

Settings (Zahnrad-Icon oben rechts). Hier liegt alles, was nicht zur Hauptbühne gehört: Service- Steuerung, Permissions, Modell-Info, CSV-Export – und genau dort findet der Nutzer auch den Button, um das Onboarding noch einmal anzusehen oder ein eigenes Modell zu trainieren.

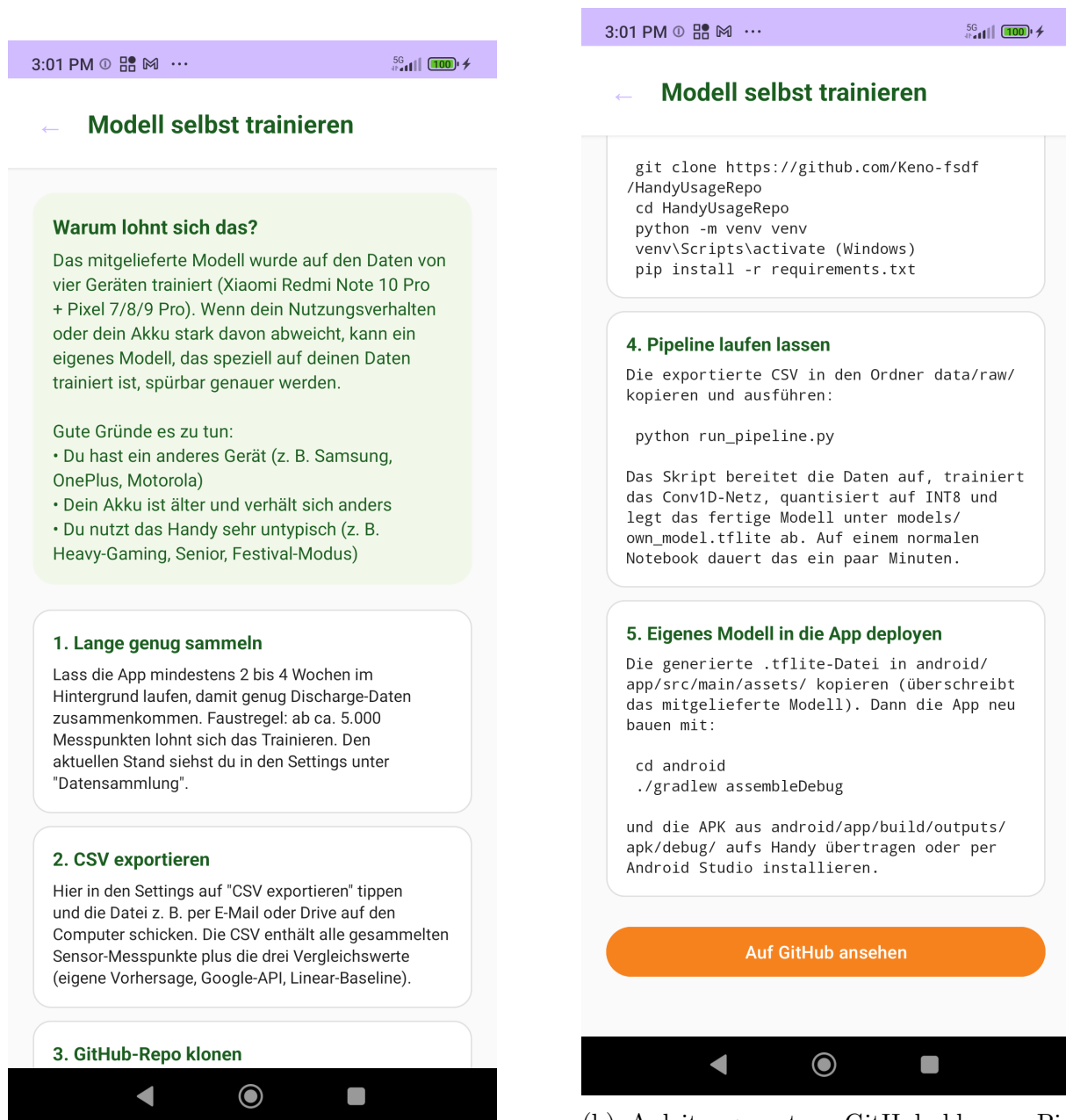


(a) Settings, oberer Teil: Service-Status mit Laufzeit und Datenpunkt-Counter, Start-/Stopp-Buttons, Berechtigungs-Buttons und Modell-Info.

(b) Settings, unterer Teil: CSV-Export, der wichtige Button „Onboarding noch einmal ansehen“ und der Einstieg in den Trainings-Guide „Modell selbst trainieren“.

Abbildung 8: Settings-Activity. Hinweis: der Button „Onboarding noch einmal ansehen“ ist exakt für den Fall gedacht, dass der Prüfer das Onboarding versehentlich weggeklickt hat.

Eigenes Modell trainieren (aus Settings erreichbar). Wer das mitgelieferte Modell durch eines ersetzen möchte, das auf den eigenen Daten trainiert ist, findet hier eine Anleitung in fünf Schritten inklusive Repo-Link.



(a) Anleitung, oben: Warum-Block plus die ersten zwei Schritte (lange sammeln, CSV exportieren).

(b) Anleitung, unten: GitHub klonen, Pipeline laufen lassen, .tflite zurück in android/app/assets/ deployen. Großer „Auf GitHub ansehen“-Button.

Abbildung 9: Train-Own-Model-Activity, aufgerufen über den Settings-Button „Modell selbst trainieren“.

Die UI im Überblick. Die folgenden Punkte fassen die zentralen UI-Eigenschaften zusammen:

- *Vier-Slide-Onboarding* beim ersten Start. Erklärt in einfachen Worten, was die App tut, und zeigt im vierten Slide einen Größen-Vergleich „Unser Modell: 14,4 KB – WhatsApp-Sprachmemo: ~ 80 KB – typisches Sprachmodell: ~ 5 GB“.
- *Tab „Live“*. Der Haupt-Tab der App. Vorhersage prominent oben, direkt darunter eine *Confidence-Pille* (grün / gelb / rot) und ein Live-Block mit allen zehn Sensoren als animierte Karten.
- *Tab „Algorithmus“*. Eigener Tab für die Erklärung des USP-Features. Ganz oben – gleich beim Öffnen sichtbar – drei Pillen mit den Alleinstellungs-Argumenten (**offline**, **14,4 KB**, **jedes Android**), darunter drei aufeinander aufbauende Visualisierungen:
 - *Datenfluss* – zehn pulsierende Sensoren links, leuchtende TinyML-Box in der Mitte, fließende Partikel hinüber zur Vorhersage rechts (high-level Überblick).
 - *Modell-Architektur* – direkt darunter die eigentlichen Conv1D-Schichten als Detail-Ansicht: Input-Matrix 10×10 , 16 Feature-Maps, 16 gemittelte Pool-Werte, ein Dense-Output. Eine sequentielle Forward-Pass-Animation lässt eine „Welle“ von oben nach unten durch die Schichten laufen, damit nachvollziehbar wird, wie das Modell intern rechnet.
 - *Live-Sparkline* der letzten drei Stunden Akkuverlauf mit gestrichelter Vorhersage-Extension für die nächste Stunde (praktische Anwendung des Modells in Echtzeit).

Danach die textuellen vier nummerierten Schritte (Sensoren $\rightarrow$ Sliding-Window $\rightarrow$ TinyML $\rightarrow$ Vorhersage) und ein USP-Block am Ende (komplett offline, 14,4 KB, jedes Android). Macht den Algorithmus selbst zu einem sichtbaren, animierten App-Element statt nur Black-Box-Output.

- *Tab „Genauigkeit“*. Zeigt einen 3-Wege-Score „Wer war heute genauer?“ mit Medail-len (eigenes Modell vs. Google-API vs. Linear-Baseline), berechnet über den letzten beobachteten Discharge-Lauf.
- *Settings hinter dem Zahnrad*. Alle Infrastruktur- Bedienelemente (Service-Status mit Laufzeit, Permissions-Check, Start/Stop, CSV-Export, Modell-Info-Karte) sind bewusst aus den Haupt-Tabs ausgelagert, damit der Nutzer beim Öffnen der App genau *ein* Feature sieht und nicht ein Feature-Buffer.
- *Erweiterte Notification*. Statt nur „Akku noch X h Y min“ steht in der Sperrbildschirm-Anzeige jetzt zusätzlich die Confidence-Pille – die Vorhersage hat eine Verlässlichkeits-Aussage immer mit dabei.

1.5 Python-Pipeline im Detail

Der Orchestrator `run_pipeline.py` bringt die TinyML-Trainings-Pipeline in vier Schritten durch:

1. `methods/tinyml/merge_devices.py` – alle Geräte-CSVs zu einer kombinierten CSV zusammenführen.
2. `methods/tinyml/data_prep.py` – Discharge-Segmente finden, Sliding-Windows bilden, Segment-Level-Split.
3. `methods/tinyml/train.py` – Conv1D-Modell trainieren.
4. `methods/tinyml/tflite_convert.py` – Konvertierung in INT8-quantisiertes TFLite (zwei weitere Varianten – dynamic range und float16 – werden parallel erzeugt und gegeneinander verglichen).

Methodisch zentral sind dabei der **Segment-Level-Split** in Schritt 2 (verhindert Data-Leakage, weil aufeinanderfolgende Messpunkte stark korreliert sind) und die **INT8-Quantisierung** in Schritt 4 (ermöglicht die 14,4 KB-Modellgröße).

Die eigentliche Genauigkeits-Auswertung gegen die Google-API und eine lineare Baseline läuft direkt in der App im Genauigkeits-Tab (siehe Abschnitt 1.4).

2 Datenmodell

2.1 CSV-Schema

Pro Messpunkt schreibt die App eine Zeile in die CSV. Die ersten drei Spalten sind Metadaten, die zehn nächsten sind die *Features* für das ML-Modell, die letzten vier sind Vergleichswerte zur Auswertung. Tabelle 3 zeigt die Spalten im Detail.

Tabelle 3: Felder eines CSV-Eintrags

Spalte	Typ	Bedeutung
session_id	String (8)	Zufalls-ID pro Service-Start
timestamp	long (ms)	Unix-Zeitstempel
datetime	String	YYYY-MM-DD HH:MM:SS
battery_level	Float 0–100	Akku-Stand in Prozent
screen_on	Int 0/1	Bildschirm an?
brightness	Float 0–100	Helligkeit normalisiert
active_app_category	Int 0–5	0=idle, 1=social, 2=video, 3=gaming, 4=browser, 5=productivity
wifi_on	Int 0/1	WiFi-verbunden?
mobile_data_on	Int 0/1	Mobilfunk aktiv?
charging	Int 0/1	Lädt gerade?
cpu_usage	Float 0–100	CPU-Frequenz-Auslastung (Proxy)
temperature	Float (°C)	Akku-Temperatur
hotspot_on	Int 0/1	WLAN-Hotspot aktiv?
system_estimate_min	Float	Google-API-Schätzung in Minuten
system_personalized	Int –1/0/1	Ist Google-Schätzung gerätspezifisch?
own_prediction_h	Float	Eigene TinyML-Vorhersage (Stunden)
linear_baseline_h	Float	Naive Vorhersage aus BatteryManager-Countern

2.2 Warum genau diese zehn Features?

Bei der Auswahl gab es eine harte Regel: *ich darf nur lesen, was eine normale App ohne Spezial-Berechtigung lesen kann*. Damit fielen einige naheliegende Kandidaten weg:

- **Genaue Stromstärke pro App** (PowerStats-API, nur für System-Apps freigeschaltet): nicht zugänglich.
- **Battery Health** (Verschleißzustand des Akkus): teilweise gesperrt.
- **Direkter Kernel-Zugriff auf Akku-Rohdaten** (Fuel-Gauge): gesperrt.

Die zehn Features gliedern sich in drei Gruppen: **Akkuzustand** (level, temperature, char-

ging), **Nutzungsintensität** (screen, brightness, app_category, cpu_usage) und **Konnektivität** (wifi, mobile, hotspot).

2.3 Speicherung und Export

Die CSV liegt im app-internen Verzeichnis (`Context.filesDir`). Das ist nur für diese App lesbar – weder andere Apps noch ein angeschlossener PC können die Datei direkt sehen (siehe Datenschutz-Subsection 2.4). Der Export erfolgt explizit per Share-Intent durch den Nutzer.

Über vier Geräte hinweg wurden in 48 Tagen **66.001 Messpunkte** in **180 Sessions** gesammelt.

2.4 Datenschutz

Was wird erhoben

Die App speichert ausschließlich technische Geräte-Metriken (die Spalten oben). Es gibt *keinen* Zugriff auf:

- Standort (keine `ACCESS_*_LOCATION`-Permission),
- Kontakte, Anrufprotokolle, SMS,
- Mikrofon, Kamera,
- Inhalte von Apps (kein Accessibility-Service, kein Screen-Recording),
- personenbezogene Identifikatoren wie IMEI oder Telefonnummer.

Das einzige potenziell sensible Feld ist `active_app_category`, und auch das nur als *Kategorie* (0–5), nicht als konkreter Paketname.

Wo gespeichert wird

Im internen Verzeichnis der App (`Context.filesDir`). Nur diese App selbst hat Zugriff; andere Apps können nichts lesen (Android-Sandbox); bei Deinstallation werden die Daten automatisch mit entfernt; die Datei wird weder in iCloud noch in Google Backup synchronisiert. Ein Export findet ausschließlich über expliziten Nutzerklick (Share-Intent) statt – es gibt keinen automatischen Upload, keinen Server und keine Telemetrie.

Berechtigungen

Tabelle 4: Im Manifest deklarierte Permissions

Permission	Wofür
FOREGROUND_SERVICE	Background-Service mit Notification
FOREGROUND_SERVICE_SPECIAL_USE	Service-Typ ab Android 14
WAKE_LOCK	Service läuft auch bei Display aus
RECEIVE_BOOT_COMPLETED	Auto-Restart nach Geräte-Neustart
ACCESS_NETWORK_STATE	WiFi-/Mobile-Daten-Status lesen
ACCESS_WIFI_STATE	WiFi-Status, Hotspot-Reflection
PACKAGE_USAGE_STATS	Aktive App lesen (Nutzer muss manuell aktivieren)
POST_NOTIFICATIONS	Vorhersage in Notification anzeigen
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Service vor Doze schützen

Was der Nutzer aktiv freischalten muss. Drei dieser Permissions kann der Nutzer Android nicht einfach durchwinken – sie brauchen ein explizites Antippen, weil sie weit mehr können als die App tatsächlich braucht. Die App führt den Nutzer per Button-Klick direkt in die jeweils richtige Systemeinstellung:

Tabelle 5: Berechtigungen mit Nutzer-Aktion

Berechtigung	Wie wird sie erteilt?	Warum braucht die App sie?
Benachrichtigungen	Standard-Dialog beim ersten Start (ab Android 13)	Foreground-Service braucht Notification, über die er sichtbar ist; ohne POST_NOTIFICATIONS startet der Service gar nicht.
Nutzungsdatenzugriff	Manuell in den Systemeinstellungen via Button „Nutzungsdatenzugriff aktivieren“	Damit die App weiss, welche App-Kategorie gerade aktiv ist (Social, Video, Spiel, ...) – eines der zehn Features. Ohne Aktivierung fällt das Feature einfach auf „Leerlauf“ zurück und die App läuft trotzdem.
Akku-Optimierung deaktivieren	Standard-Dialog via Button „Akku-Optimierung deaktivieren“	Damit der Service auch nach Stunden Inaktivität nicht von Doze oder herstellerseitigen Aggressive-Killer-Regeln (vor allem MIUI) unterbrochen wird.

Wo der Nutzer das findet. Beide Permissions-Buttons (Nutzungsdatenzugriff und Akku-Optimierung) sind im *Settings*-Screen (Zahnrad-Icon oben rechts) gebündelt – in der

Sektion „Berechtigungen“ mit den Beschriftungen „Nutzungsdatenzugriff aktivieren“ und „Akku-Optimierung deaktivieren“. Ein Tap genügt, dann landet der Nutzer direkt auf der passenden System-Einstellung (`ACTION_USAGE_ACCESS_SETTINGS` bzw. `ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`). Der Status „erteilt“ bzw. „fehlt“ wird im Settings-Screen direkt darüber angezeigt.

Hinweis zu zusätzlichen System-Einstellungen. Auf vielen Geräten reicht der Tap auf den Button „Akku-Optimierung deaktivieren“ nicht für dauerhaften Hintergrund-Betrieb aus. Je nach Hersteller und Android-Version muss der Nutzer zusätzlich manuell in den Akku-Einstellungen die App auf „Uneingeschränkt“ stellen, Auto-Start erlauben oder App-Hibernation ausschalten. Besonders ausgeprägt ist das bei OEM-Skins wie MIUI (Xiaomi), One UI (Samsung) oder ColorOS (Oppo), tritt aber auch auf Stock-Android (Pixel) teilweise auf. Diese Schritte kann die App aus Sicherheitsgründen nicht selbst übernehmen.

3 Programmierung

3.1 Datensammlung im Service

Der Kern des DataCollectorService ist ein Handler.postDelayed-Loop, der alle 30 Sekunden eine Messung auslöst:

```

1  private val INTERVAL_MS = 30 * 1000L // 30 Sekunden
2
3  private val collectRunnable = object : Runnable {
4      override fun run() {
5          collectAndLog()
6          handler.postDelayed(this, INTERVAL_MS)
7      }
8  }
9
10 private fun collectAndLog() {
11     // 1) Roh-Features lesen
12     val batteryLevel = getBatteryLevel()
13     val screenOn = if (isScreenOn()) 1 else 0
14     val brightness = getScreenBrightness()
15     // ... (10 Features insgesamt)
16
17     // 2) TinyML-Vorhersage (fail-safe)
18     val features = floatArrayOf(batteryLevel, screenOn.toFloat(), /*
19         ... */
20     var prediction = -1f
21     try {
22         predictor?.let { prediction = it.addDataAndPredict(features)
23     } catch (e: Exception) {
24         Log.e("BatteryCollector", "Prediction failed: ${e.message}")
25     }
26
27     // 3) Vergleichswerte einlesen
28     val systemEstimateMin = getSystemBatteryEstimate()
29     val linearBaselineH = getLinearBaselineHours()
30
31     // 4) Datenpunkt schreiben
32     logger.log(BatteryDataPoint(/* ... */), sessionId)

```

Quellcode 1: Mess-Schleife im DataCollectorService

3.2 Auslesen der 10 Features

Die meisten Werte kommen direkt aus den Android-System-Services. Drei Sachen waren tricky.

CPU-Auslastung

Seit Android 8 ist `/proc/stat` für Apps gesperrt. Statt der „echten“ CPU-Auslastung lese ich die aktuelle CPU-Frequenz pro Kern und vergleiche mit der Maximalfrequenz. Höhere Frequenz = höhere Last:

```
1 private fun getAvgCpuFrequencyPercent(): Float {
2     val numCores = Runtime.getRuntime().availableProcessors()
3     var totalRatio = 0f
4     var readCores = 0
5     for (i in 0 until numCores) {
6         try {
7             val cur = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              scaling_cur_freq")
8                 .readText().trim().toLong()
9             val max = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              cpuinfo_max_freq")
10                .readText().trim().toLong()
11             if (max > 0) {
12                 totalRatio += cur.toFloat() / max
13                 readCores++
14             }
15         } catch (_: Exception) { }
16     }
17     if (readCores == 0) throw Exception("No CPU freq readable")
18     return (totalRatio / readCores * 100f).coerceIn(0f, 100f)
19 }
```

Quellcode 2: CPU-Last als Frequenz-Proxy

Aktive App

Mit dem `UsageStatsManager` (Permission `PACKAGE_USAGE_STATS`, die der Nutzer manuell in den Einstellungen aktivieren muss) lese ich die zuletzt benutzte App und ordne sie über ein Package-Prefix-Mapping einer Kategorie zu:

```
1 private val APP_CATEGORIES = mapOf(
2     "com.instagram"           to 1, // social
3     "com.whatsapp"            to 1,
4     "org.telegram"            to 1,
5     "com.google.android.youtube" to 2, // video
6     "com.netflix"             to 2,
```

```

7      "com.spotify"                to 2,
8      "com.supercell"              to 3, // gaming
9      "com.miHoYo"                 to 3,
10     "com.google.android.apps.chrome" to 4, // browser
11     "org.mozilla"                 to 4,
12     "com.google.android.apps.docs"  to 5, // productivity
13     "com.microsoft"                to 5,
14     // ... (rund 30 Eintraege)
15 )

```

Quellcode 3: App-Kategorisierung (Auszug)

Unbekannte Packages bekommen Kategorie 0 (idle), werden aber zusätzlich geloggt, damit ich sie später nachmappen kann.

Hotspot-Status

Die offizielle `WifiManager.isWifiApEnabled()`-API ist als `@hide` markiert, also nicht im SDK. Über Reflection ist sie trotzdem erreichbar:

```

1 private fun isHotspotOn(): Boolean {
2     return try {
3         val wm = applicationContext.getSystemService(Context.
4             WIFI_SERVICE) as WifiManager
5         val method = wm.javaClass.getDeclaredMethod("isWifiApEnabled")
6         method.isAccessible = true
7         method.invoke(wm) as Boolean
8     } catch (e: Exception) {
9         false
10    }
11 }

```

Quellcode 4: Hotspot-Status via Reflection

3.3 On-Device-Inferenz mit dem TinyML-Modell

Der `BatteryPredictor` hält das TinyML-Modell und einen Ringpuffer mit den letzten 10 Datenpunkten. Erst wenn der Puffer voll ist, wird vorhergesagt. Wichtig: die Features müssen mit denselben `StandardScaler`-Werten normalisiert werden, mit denen das Modell trainiert wurde – die Werte sind hardcoded in der App und werden bei jedem Re-Training neu hineingeschrieben.

```

1 class BatteryPredictor(context: Context) {
2     companion object {
3         const val SEQUENCE_LENGTH = 10

```

```

4      const val NUM_FEATURES = 10
5      // Werte aus models/scaler.joblib (Multi-Device-Training)
6      private val SCALER_MEAN = floatArrayOf(
7          68.79f, 0.83f, 8.47f, 1.31f, 0.08f,
8          0.92f, 0.00f, 46.35f, 33.00f, 0.07f
9      )
10     private val SCALER_SCALE = floatArrayOf(
11         29.61f, 0.38f, 13.01f, 1.17f, 0.27f,
12         0.27f, 1.00f, 16.93f, 4.21f, 0.26f
13     )
14 }
15 private val interpreter: Interpreter
16 private val recentData = ArrayDeque<FloatArray>(SEQUENCE_LENGTH)
17
18 fun addDataAndPredict(features: FloatArray): Float {
19     // NaN/Inf-Guard vor StandardScaler
20     for (v in features) if (v.isNaN() || v.isInfinite()) return
21         -1f
22
23     val normalized = FloatArray(NUM_FEATURES) { i ->
24         val s = SCALER_SCALE[i]
25         if (s > 1e-6f) (features[i] - SCALER_MEAN[i]) / s else 0f
26     }
27     if (recentData.size >= SEQUENCE_LENGTH) recentData.
28         removeFirst()
29     recentData.addLast(normalized)
30     if (recentData.size < SEQUENCE_LENGTH) return -1f
31
32     val input = ByteBuffer.allocateDirect(SEQUENCE_LENGTH *
33         NUM_FEATURES * 4)
34         .order(ByteOrder.nativeOrder())
35     for (step in recentData) for (v in step) input.putFloat(v)
36     val output = ByteBuffer.allocateDirect(4).order(ByteOrder.
37         nativeOrder())
38     interpreter.run(input, output)
39     output.rewind()
40     return output.float.coerceAtLeast(0f)
41 }

```

Quellcode 5: Inferenz mit TFLite

3.4 Robustheit – die App muss überleben

Das war der frustrierendste Teil. Android killt Hintergrund-Services aggressiv, gerade auf Xiaomi-Geräten mit MIUI-Aufsatz. Drei Mechanismen sorgen dafür, dass die Datensamm-

lung nicht abreißt:

1. **Foreground-Service** mit dauerhafter Notification.
2. **Auto-Restart nach Wegwischen** der App-Karte. Wenn der Nutzer die App in der Übersicht wegwischt, registriert sich der Service über `AlarmManager.setExactAndAllowWhileIdle()` einen Restart in 1 Sekunde.
3. **Auto-Restart nach Geräte-Neustart** über einen `BootReceiver`, der das `BOOT_COMPLETED`-Intent abfängt – aber nur, wenn der Nutzer den Sammler explizit aktiviert hatte.

```

1  override fun onTaskRemoved(rootIntent: Intent?) {
2      val restartIntent = Intent(this, DataCollectorService::class.java
3          )
4      val pendingIntent = PendingIntent.getService(
5          this, 99, restartIntent,
6          PendingIntent.FLAG_ONE_SHOT or PendingIntent.FLAG_IMMUTABLE
7      )
8      val alarm = getSystemService(Context.ALARM_SERVICE) as
9          AlarmManager
10     alarm.setExactAndAllowWhileIdle(
11         AlarmManager.ELAPSED_REALTIME_WAKEUP,
12         SystemClock.elapsedRealtime() + 1000,
13         pendingIntent
14     )
15     super.onTaskRemoved(rootIntent)
16 }

```

Quellcode 6: Auto-Restart bei onTaskRemoved

3.5 Schema-Drift im Logger

Beim Hinzufügen neuer Spalten (etwa `system_personalized` oder `linear_baseline_h`) hatten bestehende CSV-Dateien weniger Spalten als die neue Version erwartete. Der Logger prüft deshalb beim Start die Header-Zeile und parkt alte CSVs automatisch:

```

1  init {
2      val expectedHeader = "session_id,timestamp,datetime,{
3          BatteryDataPoint.CSV_HEADER}"
4      if (!csvFile.exists() || csvFile.length() == 0L) {
5          csvFile.writeText("$expectedHeader\n")
6      } else {
7          val firstLine = csvFile.useLines { it.firstOrNull() ?: "" }
8          if (firstLine != expectedHeader) {
9              val backup = File(context.filesDir,
10                  "battery_data_legacy_${System.currentTimeMillis()}.
11                  csv")

```

```

10         csvFile.copyTo(backup, overwrite = true)
11         csvFile.writeText("$expectedHeader\n")
12     }
13 }
14 }

```

Quellcode 7: Schema-Drift im Logger abfangen

3.6 Python-Pipeline – Beispiel Segment-Erkennung

Die wichtigste Pipeline-Funktion findet Entlade-*Segmente*: zusammenhängende Phasen, in denen das Phone entladen wird, ohne zwischendurch zu laden. Daraus werden später Sliding-Window-Sequenzen gebildet.

```

1 def find_discharge_segments(df, min_length, max_gap_ms):
2     segments = []
3     for _, session_df in df.groupby("session_id"):
4         session_df = session_df.sort_values("timestamp").reset_index(
5             drop=True)
6         is_discharging = session_df["charging"] == 0
7         block_id = (~is_discharging).cumsum() # neuer Block bei
8             Laden
9         for _, block in session_df[is_discharging].groupby(block_id[
10             is_discharging]):
11             # innerhalb des Blocks bei groesseren Zeitluecken
12             splitten
13             time_diffs = block["timestamp"].diff()
14             gap_id = (time_diffs > max_gap_ms).cumsum()
15             for _, sub in block.groupby(gap_id):
16                 if len(sub) >= min_length:
17                     segments.append(sub.reset_index(drop=True).copy())
18             )
19     return segments

```

Quellcode 8: Discharge-Segmente finden

3.7 Trainings-Verlauf

Abbildung 10 zeigt den Verlauf von Trainings- und Validation-MAE (durchschnittliche Abweichung der Vorhersage in Stunden) des TinyML-Modells während der 21 Epochen, nach denen EarlyStopping greift. Trainings- und Validation-Kurve liegen über den gesamten Verlauf nah beieinander und sinken gemeinsam ab – das ist das gewünschte Signal dafür, dass das Modell nicht überanpasst (Overfitting), sondern die gelernten Muster auf bisher ungesehene Daten überträgt.

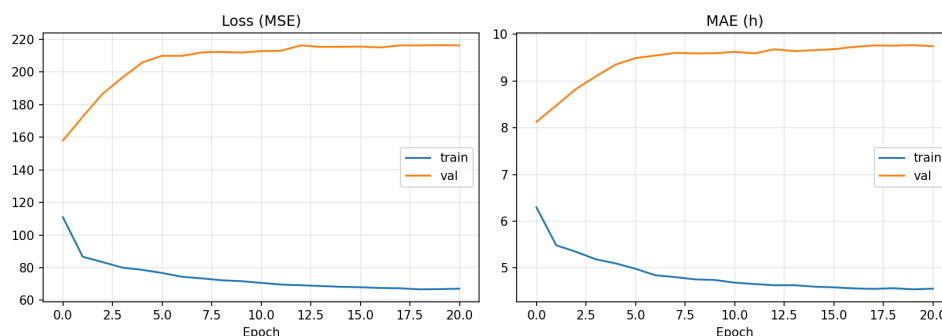


Abbildung 10: Trainings- und Validation-MAE (durchschnittliche Abweichung in Stunden) des TinyML-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5h. Da EarlyStopping mit `restore_best_weights=True` konfiguriert ist, wird das beste Modell zurückgegeben.

3.8 Code-Umfang und Eigenleistung

Das Projekt umfasst drei parallel entstandene Schichten: die Android-App in Kotlin, die ML-Pipeline am PC und die Auswertungs-Schicht.

Tabelle 6: Code-Umfang nach Bereich

Bereich	Zeilen (ohne Leerzeilen)
Android-App Kotlin (20 Dateien)	≈ 1.450
Android-App Ressourcen (XML-Layouts + Drawables)	≈ 650
Python-Pipeline (<code>methods/</code> , 13 Module)	≈ 1.150
Python-Auswertung (<code>evaluation/</code> , 12 Module)	≈ 2.050
Konfiguration und Orchestrator	≈ 300
Summe	≈ 5.600

Eigene Entwurfs- und Methodik-Entscheidungen:

- Gesamte Android-App-Struktur (Activity, Service, BroadcastReceiver, FileProvider), inklusive `onTaskRemoved`-Auto-Restart und Schema-Drift-Detection.
- Auswahl und Implementierung der zehn Features (CPU-Frequenz-Proxy, App-Kategorie-Mapping, Hotspot-Reflection).
- Datenmodell, CSV-Schema und FileProvider-Konfiguration für den Share-Export.
- Pipeline-Architektur (modulare Trennung von Trainings- und Auswertungs-Schicht).
- Konkrete Implementierung der linearen Baseline als Vergleichswert zur TinyML-Vorhersage.

- Segment-Level-Split als Anti-Leakage-Mechanismus.
- Auswertungs-Skripte: Genauigkeits-Metriken, Per-Device-Analyse, Daten-Diagnose.

Übernommen:

- Conv1D-Architektur in Anlehnung an gängige TinyML-Topologien [1, 3] (zwei Convolutions, GlobalAveragePooling, zwei Dense).
- TFLite-Konvertierungs-API (Standard-Funktionen von TensorFlow).
- `scikit-learn`: `StandardScaler`, `train_test_split`.

Probleme und wie sie gelöst wurden:

- *Service stirbt auf MIUI*: Auto-Restart über AlarmManager.
- */proc/stat gesperrt*: Frequenz-Proxy statt echter Auslastung.
- *CSV-Schema-Migration*: Header-Check beim Service-Start.
- *TinyML auf Single-Device zeigt MAE 0.5h Artefakt*: Segment-Level-Split eingeführt, korrigiert den scheinbaren Wert auf 9.96 h.

3.9 Reproduzierbarkeit der Pipeline

Damit die Auswertung nicht von Hand-Schritten abhängt, gibt es einen einzigen Orchestrator `run_pipeline.py`, der die komplette Verarbeitungskette ausführt – vom Mergen der Geräte-CSVs bis zu den fertigen Auswertungs-Reports.

```
1 # vollstaendige Pipeline: ca. 20 Minuten auf Laptop-CPU
2 python run_pipeline.py
3
4 # nur die ML-Schritte (ohne Eval) -- ca. 5 Minuten
5 python run_pipeline.py --stage tinyml
```

Quellcode 9: Pipeline reproduzierbar starten

Alle Hyperparameter, Feature-Listen, Random-Seeds und Pfade liegen in `configs/default.yaml`. Beim Re-Training werden zusätzlich die `StandardScaler`-Werte für die Android-App neu exportiert (`methods/tinyml/export_scaler_for_android.py`) und ein Check-Script vergleicht sie mit den hardcoded Werten in `BatteryPredictor.kt`, damit die Normalisierung in App und Modell garantiert übereinstimmt.

4 Abschließender Erfahrungsbericht

4.1 Wöchentliche Einträge

Woche 1 (KW 13: 23.–29. März)

Setup und erste Datensammlung. In dieser Woche das Repo angelegt und das Android-Projekt aufgesetzt. Erste Version des `DataCollectorService` mit Claude geprototyp. Innerhalb der Woche lief die erste Messung auf dem Xiaomi durch. Hauptproblem in dieser Woche: die `PACKAGE_USAGE_STATS`-Permission ist nicht via Standard-Dialog vergebbar – der Nutzer muss in den Einstellungen manuell ankreuzen. Ich habe deshalb einen Button in der App eingebaut, der direkt zum `ACTION_USAGE_ACCESS_SETTINGS` springt.

Woche 2 (KW 14: 30. März – 5. April)

Service-Robustheit – und parallel Statistik-Grundlagen aufgefrischt. Datensammlung bricht beim Wegwischen aus der App-Übersicht ab. Drei Tage gebraucht, um herauszufinden, wie man auf Xiaomi-MIUI einen Service vor dem Aggressive-Killing schützt. Lösung: `onTaskRemoved` → `AlarmManager.setExactAndAllowWhileIdle` → Service kommt 1 Sekunde später wieder. Plus `BootReceiver` damit nach Neustart automatisch weitergesammelt wird.

Parallel dazu Statistik-Grundlagen aufgefrischt, die für die spätere Modell-Auswertung relevant werden: Cross-Validation, Train/Test/Validation-Splits, Data-Leakage-Mechanismen und Confounder-Erkennung.

Woche 3 (KW 15: 6.–12. April)

Erstes Training – und sofort verdächtig. Mit rund 11.000 Messpunkten lief das erste Training. Das Modell zeigte einen MAE (durchschnittliche Abweichung der Vorhersage in Stunden) von nur 0,53 h – deutlich besser als die Google-API. Beim Nachprüfen stellte sich heraus, dass dieser Wert ein Artefakt war: der zufällige Train/Test-Split hatte aufeinanderfolgende Messpunkte auf beide Sets verteilt, sodass die Trainingsdaten praktisch die Testdaten vorwegnahmen (Data Leakage). Nach Umstellung auf einen Segment-Level-Split – bei dem ganze Discharge-Phasen entweder komplett im Trainings- oder komplett im Test-Set landen – lag der MAE realistisch bei 9,96 Stunden.

In dieser Phase eng mit Claude gearbeitet: jede Pipeline-Änderung wurde gegen die vorherigen Ergebnisse geprüft. Am 12. April App-APK an ein Familienmitglied mit Pixel 7 Pro weitergegeben.

Woche 4 (KW 16: 13.–19. April)

Multi-Device wird real. APK an zwei weitere Familienmitglieder verteilt (Pixel 8 Pro, Pixel 9 Pro XL); damit lief die Datensammlung auf vier Geräten (eigener Xiaomi plus drei Familien-Pixels). Vorher jeweils erklärt, was die App sammelt (Akkustand, App-Kategorie und ähnliche Geräte-Metriken, alle Daten bleiben lokal). Auf den Pixel-Geräten hat es länger gedauert, den Service stabil zum Laufen zu bringen, weil die Auto-Restart-Logik aus Woche 2 pro Hersteller-Variante leicht angepasst werden musste. Parallel das CSV-Schema erweitert (`system_personalized`, `linear_baseline_h`) und den Logger so gebaut, dass er bei Schema-Wechsel die alte Datei sicher parkt.

Woche 5 (KW 17: 20.–26. April)

Auswertungs-Tiefe. Pipeline so umgebaut, dass sie alle vier Geräte-CSVs konsolidiert und auswertet. Die lineare Baseline als Vergleichswert zur TinyML-Vorhersage implementiert – damit zeigt der Genauigkeits-Tab in der App live drei Vorhersage-Quellen (eigenes TinyML-Modell, Google-API und lineare Baseline) mit ihrem jeweiligen MAE im letzten Discharge-Lauf.

Woche 6 (KW 18: 27. April – 3. Mai)

Auswertungs-Infrastruktur fertig stellen. Auswertungs-Skripte für Modell-Genauigkeit, Gerät-Vergleiche und Datenverteilungs-Diagnose in die Pipeline integriert.

Woche 7 (KW 19–20: 4.–11. Mai)

Konsolidierung, UI-Refactor und Dokumentation. Datensammlung beendet (10. Mai). Finale Datenmenge: 66.001 Messpunkte über vier Geräte. In den letzten Tagen UI-Refactor: Vier-Slide-Onboarding, drei Tabs (Live / Algorithmus / Genauigkeit), Infrastruktur-Buttons hinter ein Zahnrad-Icon ausgelagert, Confidence-Pille in Live-Tab und Notification. Rest der Woche: Portfolio + Pipeline-Konsistenz-Checks + Erfahrungsbericht.

Woche 8 (KW 20: 12.–18. Mai)

Polish nach Abgabe-Phase. Die Doku am Stück gegengelesen und einige Stellen mit zu plakativer Sprache nüchterner umformuliert. Kleinere App-Bugfixes (UI-Glitches im LiveFragment, ein CSV-Header-Edge-Case beim Schema-Wechsel zwischen alter und neuer Spaltenanordnung).

Woche 9 (KW 21: 19.–25. Mai)

Doku-Schliff Section 0 (Alleinstellungsmerkmal und Problemstellung). Den USP-Abschnitt der Doku komplett überarbeitet – das eine Hauptfeature (TinyML on-

device, vom Nutzer nachtrainierbar) klar nach vorne gerückt und betont.

Woche 10 (KW 22: 26. Mai – 1. Juni)

Wenig am Projekt, hauptsächlich für andere Fächer gelernt. Diese Woche kaum am Portfolio weitergekommen – die Klausurvorbereitung für andere Fächer hat den größten Teil der Zeit beansprucht. Am Projekt nur ein paar kleinere Wording-Korrekturen am Rande.

Woche 11 (KW 23: 2.–8. Juni)

Architektur-Grafik renoviert. Die alte Gesamtarchitektur-PNG im Portfolio aktualisiert, neu aus der überarbeiteten `architektur_gesamt.puml` via `plantuml.jar` gerendert. Parallel weiter für andere Fächer gelernt.

Woche 12 (KW 24: 9.–15. Juni)

Klausurvorbereitung. Für Klausurfächer gelernt, nichts am Portfolio gemacht.

Woche 13 (KW 25: 16.–22. Juni)

Klausurvorbereitung weiter, kleinere Doku-Korrekturen. Parallel zur Klausurvorbereitung anderer Fächer ein paar Konsistenz- und KI-Halluzinations-Korrekturen an der Portfolio-Dokumentation.

Woche 14 (KW 26: 23.–29. Juni)

Klausurvorbereitung. Für Klausurfächer gelernt, nichts am Portfolio gemacht.

Woche 15 (KW 27: 30. Juni – 6. Juli)

Finaler Doku-Korrekturdurchgang. Letzte vollständige Lese-Runde der Portfolio-Doku mit Fokus auf Tippfehler und finale Konsistenz-Checks. Klausurvorbereitung anderer Fächer parallel.

Woche 16 (KW 28: 7.–12. Juli)

Klausurvorbereitung. Nur noch für die Klausuren gelernt.

4.2 Schlüssel-Entscheidungen, Fehler und Trade-offs

Tabelle 7 listet die zentralen Entscheidungen des Projekts mit der jeweils verworfenen Alternative und einer kurzen Begründung. Die Reihenfolge ist grob chronologisch von der Konzeption bis zur Abgabe-Woche.

Tabelle 7: Schlüssel-Entscheidungen und verworfene Alternativen

Wann	Was gewählt	Alternative verworfen	Warum so
Konzeption	Eigene Daten + on-device Modell	Vorhandener Datensatz, Cloud-Modell	Liefert echten Real-World-Bezug und volle MLOps-Schleife
W1	10 Features pro Messpunkt	3–4 „sichere“ oder 20+	Kompromiss aus Vollständigkeit und Speicher-Overhead
W5	INT8-Quantisierung (14,4 KB)	Float32-Modell (~ 50 KB)	TFLite-INT8 reicht in der Genauigkeit, ist deutlich kleiner
W7	Drei Tabs + Settings mit klarem Hauptfeature	Single-Screen mit allen Funktionen	Feature-Buffer verwirrt; klare Struktur liest sich besser
W7	Architektur-Heatmap als skizzierte Pulsation	Input-Layer live mit echten Sensor-Werten fütern	Echte Werte änderten sich über 5 Min kaum – horizontale Streifen statt aussagekräftige Heatmap, visuell schlechter
Durchgängig	KI-Modus zweigeteilt	Nur Desktop-App-Modus	Container kapselt Risiko für massive Auto-Refactors

4.3 KI-Nutzung

Während des Projekts habe ich **Claude Code** (Anthropic, Modell Opus 4.7 mit 1M-Token-Kontext) als Programmier- und Schreibassistent verwendet.

Wofür Claude genutzt wurde:

- *Code* – Boilerplate, Refactoring und Auswertungs-Skripte nach meinen Vorgaben.
- *Dokumentation* – Verfassen, Umformulieren und Strukturieren großer Teile dieses Portfolios.
- *Recherche* – Suche nach relevanter Android- und ML-Literatur.
- *Methodische Kontrolle* – systematische Audit-Runden über Code und Dokumentati-

on (Konsistenz-Checks, methodische Schwächen, Confounder-Verdacht, KI-Halluzinationen).

- *Versionsverwaltung* – Commits und Pushes wurden gemeinsam mit Claude orchestriert. Jeder Commit trägt einen **Co-Authored-By: Claude**-Footer; die KI-Beteiligung pro Änderung ist damit direkt im Git-Verlauf des öffentlichen Repos (`Keno-fsdf/HandyUsageRepo`) nachvollziehbar – nicht nur in dieser Selbstauskunft.

Was ich selbst gemacht habe:

- Das gesamte Projekt-Konzept und die App-Idee.
- Auswahl der zehn Features (was eine normale App ohne Spezial-Rechte überhaupt lesen darf).
- Datensammlung auf den vier Geräten – inklusive App-Verteilung an drei Familienmitglieder.
- Alle methodischen Entscheidungen: Feature-Auswahl, Trainingsdaten-Aufbereitung, Auswahl der in der App angezeigten Vergleichswerte.
- Kritisches Hinterfragen der KI-Vorschläge und Korrektur eigener Fehler (z. B. inkonsistente Zahlen, KI-Halluzinationen bei Quellen-Autoren, zu plakative Aussagen).
- Geräte-spezifisches Debugging (MIUI-Service-Stirbt-Problem) und alle Anpassungen am Android-Manifest sowie Permissions.

Setup und Sicherheits-Rahmen. Die KI lief in zwei Modi, je nach Risiko der Aufgabe: einem mit manueller Bestätigung jeder Datei- und Shell-Aktion (für die meisten Aufgaben am Haupt-Code) und einem Sandbox-Container mit Auto-Approve und eingeschränktem Datei- und Netzwerk-Zugriff (für riskantere oder experimentelle Aufgaben). Genutzt wurde **Claude Pro** (Subscription), nicht der API-Zugang.

Bilanz. Realistisch eingeschätzt: ohne KI-Unterstützung hätte ich das Projekt in dieser Tiefe nicht geschafft. Besonders wertvoll waren Code-Beschleunigung, Recherche und die methodischen Konsistenz-Checks über große Code- und Doku-Mengen. Was die KI *nicht* ersetzt hat: die Entscheidung, was gemacht wird, das Verstehen der Ergebnisse, das Verteidigen von Trade-offs und vor allem die handfeste Geräte-Logistik und das Debugging vor Ort.

Repository: <https://github.com/Keno-fsdf/HandyUsageRepo>

Keno Schürger

Literatur

- [1] Norah N. Alajlan and Dina M. Ibrahim. TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications. *Micromachines*, 13(6):851, 2022.
- [2] Android Open Source Project. `PowerManager.getBatteryDischargePrediction()`, 2021. Android API level 31 (Android 12).
- [3] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. MLPerf Tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.